

Relatorio - Processamento Distribuido de Logs com MPI

Aluno: Ryan Ledo

RA: 10352727

Disciplina: Computacao Distribuida

Laboratorio: Lab 2

1. Objetivo

O objetivo deste laboratorio foi desenvolver um programa em Python utilizando a biblioteca `mpi4py` para processar um grande conjunto de logs de forma distribuida. O dataset foi gerado em memoria pelo processo master e dividido entre os processos MPI utilizando a operacao coletiva `Scatter`.

Cada processo analisou somente a sua parte dos dados e contou os registros com status `404` ou `500`. Ao final, cada processo enviou ao master uma mensagem contendo seu rank, a quantidade de linhas analisadas e a quantidade de erros encontrados.

2. Tecnologias utilizadas

- Python 3
- `mpi4py`
- OpenMPI
- Docker
- Ubuntu 22.04

3. Implementacao

O processo de execucao foi dividido nas seguintes etapas:

1. O processo 0 gerou 100.000 logs aleatorios em memoria.
2. Cada log foi criado no formato:

```
IP METODO ENDPOINT STATUS
```

3. O dataset foi dividido em partes iguais. Como foram utilizados quatro processos, cada processo recebeu 25.000 linhas.
4. A distribuicao foi feita com `comm.scatter(..., root=0)`.
5. Cada processo separou os campos de cada log e verificou o codigo de status.
6. Os status `404` e `500` foram contabilizados como erros.
7. Os processos workers enviaram seus resultados ao master usando `comm.send()`.

8. O processo master recebeu as mensagens usando `comm.recv()` e imprimiu os resultados.

O programa não utiliza `Gather` ou `Reduce`, conforme especificado na atividade. A comunicação coletiva foi utilizada somente para distribuir as partes do dataset, enquanto os resultados foram enviados por mensagens ponto a ponto.

4. Execução

A imagem Docker foi criada com o comando:

```
docker build -t lab2-mpi -f .\Lab_2\Dockerfile .\Lab_2
```

Em seguida, o programa foi executado com:

```
docker run --rm lab2-mpi
```

5. Resultado obtido

A saída produzida pelo programa foi:

```
Gerando dataset de logs...
```

```
Processo 0 analisou 25000 linhas e encontrou 9939 erros.  
Processo 1 analisou 25000 linhas e encontrou 10088 erros.  
Processo 2 analisou 25000 linhas e encontrou 9994 erros.  
Processo 3 analisou 25000 linhas e encontrou 9878 erros.
```

Resumo da execução:

Processo	Linhas analisadas	Erros encontrados
0	25.000	9.939
1	25.000	10.088
2	25.000	9.994
3	25.000	9.878
Total	100.000	39.899

6. Análise dos resultados

Os resultados mostram que a distribuição foi realizada corretamente, pois os quatro processos analisaram exatamente 25.000 linhas cada. A soma das linhas processadas corresponde ao total de 100.000 logs gerados pelo processo 0.

Foram encontrados 39.899 erros no total. Como os status foram escolhidos aleatoriamente, e os codigos 404 e 500 representam dois dos cinco valores possiveis da lista de status, e esperado que aproximadamente 40% dos logs sejam classificados como erros. O resultado obtido corresponde a essa expectativa.

As quantidades de erros variam entre os processos porque os logs foram gerados aleatoriamente. Essa variacao nao indica falha na distribuicao; o importante e que cada processo recebeu a mesma quantidade de linhas e processou exclusivamente a sua propria particao.

7. Print da execução na minha máquina

```
(venv) PS C:\Users\Ryan\Documents\Reposit-rio-para-os-laborat-rios-de-Computa-o-Distribu-da\Lab_2> docker build -t lab2-mpi .
[+] Building 1.6s (10/10) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 469B
=> [internal] load metadata for docker.io/library/ubuntu:22.04
=> [auth] library/ubuntu:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/4] FROM docker.io/library/ubuntu:22.04@sha256:2eddb5dc405e9612ba3584ce95480277e3eb374407b5505fe26f17df77c7dbc
=> => resolve docker.io/library/ubuntu:22.04@sha256:2eddb5dc405e9612ba3584ce95480277e3eb374407b5505fe26f17df77c7dbc
=> [internal] load build context
=> => transferring context: 29B
=> CACHED [2/4] RUN apt update && apt install -y python3 python3-pip openmpi-bin libopenmpi-dev && python3 -m pip install --no-cache-dir mpi4py &&
=> CACHED [3/4] WORKDIR /app
=> CACHED [4/4] COPY main.py /app/main.py
=> exporting image
=> => exporting layers
=> => exporting manifest sha256:fffe298ccd9deeb5fcc89d83ea4365f41a9f30d8e32f2df8258add18ea6839d0
=> => exporting config sha256:a292fc0c08085ecb30b2a9a70e784f4cf5705959960ab3c4c5bc6d3ff2f2e4
=> => exporting attestation manifest sha256:30e6a65aa11aaec8887cd36d55744ebe2a46b188ed604cdb994a8848e2798bb
=> => exporting manifest list sha256:bc0fb332e60759e71788119df3a1848a05fc5e2b3ec50ba72c8a7e7d4c2e8a05
=> => naming to docker.io/library/lab2-mpi:latest
=> => unpacking to docker.io/library/lab2-mpi:latest
(venv) PS C:\Users\Ryan\Documents\Reposit-rio-para-os-laborat-rios-de-Computa-o-Distribu-da\Lab_2> docker status
docker: unknown command: docker status

Run 'docker --help' for more information
(venv) PS C:\Users\Ryan\Documents\Reposit-rio-para-os-laborat-rios-de-Computa-o-Distribu-da\Lab_2> docker run --rm lab2-mpi

Gerando dataset de logs...

Processo 0 analisou 25000 linhas e encontrou 9876 erros.
Processo 1 analisou 25000 linhas e encontrou 9985 erros.
Processo 2 analisou 25000 linhas e encontrou 10035 erros.
Processo 3 analisou 25000 linhas e encontrou 9951 erros.
(venv) PS C:\Users\Ryan\Documents\Reposit-rio-para-os-laborat-rios-de-Computa-o-Distribu-da\Lab_2>
```

8. Conclusao

O programa atendeu aos requisitos da atividade. Foi utilizado Python com `mpi4py`, o dataset foi gerado em memoria pelo processo 0, a distribuicao foi feita obrigatoriamente com `Scatter`, cada processo analisou somente sua parte dos dados e os resultados foram enviados ao master em mensagens individuais.

A execucao em Docker tambem permitiu reunir Python, `mpi4py` e OpenMPI em um ambiente controlado, garantindo que o programa fosse executado com quatro processos MPI.